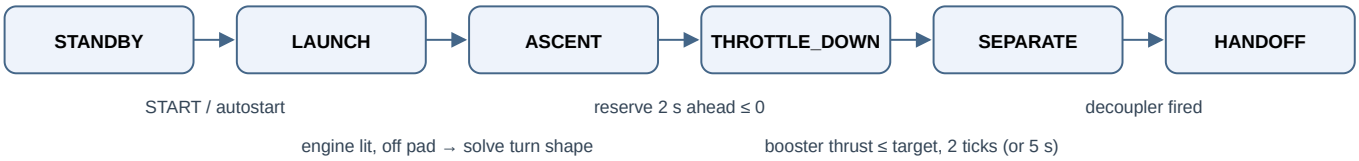
ascent: how it works

Flies a two-stage stack from the pad. MechJeb's classic ascent steers and throttles; this program chooses the turn shape for the craft, drops the booster at the last moment it can still fly home, and follows the upper stage to a circular orbit (`ORBIT_ALT_M`, 80 km). The booster is flown afterwards from FMRS's separation save by `boosterland/fmrs.py` (or, with `HANDOFF=booster`, in this process by boosterland, the upper stage abandoned).

Phases



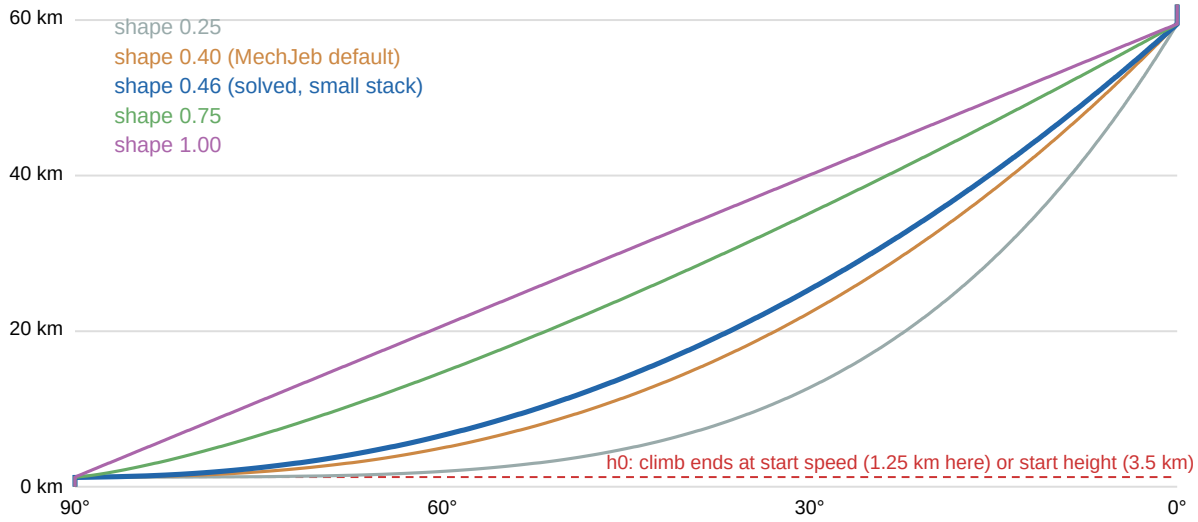
HANDOFF upper: MechJeb to orbit (default) · booster: boosterland in-process

phase	what it does	leaves when
STANDBY	sets MechJeb's ascent: classic, target 80 km, inclination, roll = the pad roll against a pitch-over to heading 90, autostage off; engages it	START pressed (or autostart)
LAUNCH	stages until an engine is lit and the vehicle is off the pad, never a stage that decouples anything. At liftoff: reads the booster from the staging, logs the upper stage's **control source** (warns if there is no crewed command part and no probe core outside the booster) and any upper-stage **autostruts to a booster part** (Heaviest/Root -- KSP tears them at separation); solves the turn shape	lit and off the pad
ASCENT	MechJeb flies; every tick `budget.recovery` computes the propellant the booster would land with if it separated now	that reserve, projected `SEP_LEAD_S` ahead, falls to `RESERVE_FRACTION` 0.15 of what the return needs
THROTTLE_DOWN	booster engine thrust limits (outer engines first) so its free acceleration is `SEP_ACCEL_RATIO` 0.75 of the upper stage's; MechJeb keeps the main throttle	delivered booster thrust under target on 2 ticks, or 5 s
SEPARATE	fires the booster's decoupler (the upper engines light with it), sets the booster's throttle to 0; its limits stay on	at once
HANDOFF (upper)	first tick: re-reads MechJeb's ascent settings and puts back any that differ; MechJeb autostage on; MechJeb burns to the apoapsis, coasts, circularizes	MechJeb disengages (logs orbit and mass), the upper stage is lost (mass 0 / landed / splashed), or `ORBIT_TIMEOUT_S`

The turn shape

MechJeb classic climbs vertically until **either** the altitude reaches its turn-start height **or** the surface speed reaches its turn-start speed (both from MechJeb's automatic path: on Kerbin 3.5 km, and a speed MechJeb derives for the craft -- 98.9 m/s on the shuttle stack). The height h0 at which the climb ended is the start of the curve:

$$\texttt{pitch} = 90 - ((h - h0) / (\text{end} - h0))^{\text{shape}} \times (90 - \text{end angle})$$, end 59.5 km, end angle 0°.



A low shape turns early and flat -- the booster ends up lower and further downrange; a high shape climbs steeply and turns late. A flat turn costs drag and sends the booster further out, so it must keep more propellant for the trip back and separates earlier; a steep turn loses to gravity. Because separation is decided by the booster's own return budget, both costs show up in one number: **the upper stage's mass in the target orbit**. `turnshape.solve` maximizes it.

read from the game at liftoff

booster masses, thrust, Isp, radius, flip authority; upper mass, LFO, thrust, Isp, lit?; Cd·A table (6 heights × 10 speeds, zero AoA); pressure and density every 1 km; MechJeb's start height, start speed, end height, end angle

fly(shape)

point mass over the rotating body; vertical until start height or start speed, then pitch(h, shape) from h0; drag against the air-relative velocity; separate when `budget.recovery` (downrange along the ground) says reserve ≤ `RESERVE_FRACTION`; upper burns to apoapsis = target; circularize: $m \cdot e^{-\Delta v / Isp\ g0}$

solve

scan shape 0.1–1.0 (11), golden section around the best to 0.01; maximum mass in orbit → `turn_shape_exponent`; no orbit at any shape: `TURN_SHAPE`

Left out of the model: angle of attack, the booster throttle-down, drag in the coast. The solve takes about half a second. Its result and the predicted mass curve are logged as `turn shape: solved ...`, and the predicted flight every `SHAPE_TRACK_S` (10 s) as `turn shape track: t:h/downrange/speed`; `ascent/tools/shapetrack.py LOG...` prints it against the flight's telemetry and the predicted against the flown separation and orbit mass.

Rules the code keeps

- The throttle is MechJeb's: the program never writes or reads `control.throttle` on the ascending vessel (kRPC would hold a written value over MechJeb); the throttle-down reads the booster engines' delivered/allowed thrust.
- The booster keeps its throttle-down limits after separation; whoever flies it next owns them (boosterland's cluster for several engine groups; `fmrs.py` clears them for one).
- What the craft must provide and the program cannot change -- a control source on the upper stage, autostruts that stay inside each stage -- it checks at liftoff and logs as a WARNING.
- Every tunable is in `ascent/config.py`; every log carries the config differences and the defaults fingerprint.